Focus Tracker Pro - README

🎓 Focus Tracker Pro

AI-Powered Student Attention Monitoring System Department of Information Technology

Focus Tracker Pro is a Flask-based web app that lets class advisors log in, start a live camera session, detect student focus/distraction using OpenCV face detection, log alerts, and generate/download PDF session reports.

✨ Features

- **Staff Login** — Class-wise login for advisors (IT-A, IT-B, IT-C)
- **Live Monitoring** — Webcam feed with real-time face detection (OpenCV Haar Cascade)
- **Alert Logging** — Automatically flags `DISTRACTED`, `SLEEPING`, and `PHONE_USE` events
- **Student Management** — Add/view/delete students per class with roll numbers
- **Session Reports** — View all past sessions with alert breakdowns
- **PDF Export** — Download a formatted session report (via ReportLab)
- **Responsive Dark-Themed UI** — Built with plain HTML/CSS + Jinja2 templates

---

🛠 Tech Stack

Layer	Technology
Backend	Flask (Python)
Database	SQLite (`focustracker.db`)
Computer Vision	OpenCV (`opencv-python-headless`)
PDF Generation	ReportLab
Frontend	HTML5, CSS3, Jinja2 templates
Deployment	Render / Heroku-style (`Procfile`)

📁 Project Structure

```
focus-tracker-pro/
├── app.py                # Main Flask application & routes
├── database.py           # DB schema + initial staff seed data
├── focustracker.db       # SQLite database file
├── requirements.txt      # Python dependencies
├── Procfile              # Deployment start command
└── templates/
```

```
    ├── home.html            # Landing page
    ├── login.html           # Staff login
    ├── forgot_password.html # Password recovery info
    ├── dashboard.html       # Advisor dashboard
    ├── students.html        # Student list management
    ├── reports.html         # Session reports list
    └── session.html         # Live monitoring session view
```

⚙ Installation & Setup

1. Clone / copy the project

```
git clone <your-repo-url>
cd focus-tracker-pro
```

2. Create a virtual environment (recommended)

```
python -m venv venv
source venv/bin/activate      # On Windows: venv\Scripts\activate
```

3. Install dependencies

```
pip install -r requirements.txt
```

4. Run the app

```
python app.py
```

The app initializes `focustracker.db` automatically on first run (via `init_db()` in `database.py`) and seeds it with the three default staff accounts.

By default it runs on `http://0.0.0.0:5000` (or the `PORT` env variable if set).

🔑 Default Login Credentials

Class	Advisor	Password
IT-A	Sathiyapriya Mam	`Sathiya@ITA`
IT-B	Gowsalya Mam	`Gowsalya@ITB`
IT-C	Sangeetha Mam	`Sangeetha@ITC`

🌐 Routes / Endpoints

Route	Method	Description

`/`	GET	Home / landing page
`/login`	GET, POST	Staff login
`/logout`	GET	Clear session
`/forgot-password`	GET	Shows advisor password info
`/dashboard`	GET	Advisor dashboard (requires login)
`/students`	GET	List students of logged-in advisor's class
`/students/add/<class_name>`	POST	Add a new student
`/students/delete/<student_id>`	GET	Delete a student
`/start-session`	GET, POST	Start a monitoring session + camera
`/video-feed`	GET	MJPEG live video stream with face detection
`/stop-session`	POST	Stop session, release camera
`/add-alert`	POST	Log a distraction/sleep/phone-use alert
`/get-stats/<session_id>`	GET	JSON stats for a session
`/reports`	GET	List all sessions for the advisor
`/download-report/<session_id>`	GET	Download PDF report for a session

🗄 Database Schema

- **staff** — `id, name, class_name, password`
- **students** — `id, register_no, name, class_name`
- **sessions** — `id, staff_id, class_name, start_time, end_time, date`
- **alerts** — `id, session_id, register_no, student_name, alert_type, time`

🚀 Deployment

The included `Procfile` (`web: python app.py`) supports deployment on platforms like **Render** or **Heroku**.

- Set the `PORT` environment variable (most platforms do this automatically).
- Since the camera (`cv2.VideoCapture`) relies on local hardware, **live camera capture will only work when running locally** — cloud deployments won't have webcam access. The `/video-feed` and `/start-session` routes are best run on a local machine or a machine with a connected camera.

⚠ Security Notes (recommended before going live)

A few things worth hardening before real deployment:

1. **Passwords are stored in plain text** in the `staff` table and shown in plain text on the `/forgot-password` page. Consider hashing passwords (e.g., with `werkzeug.security.generate_password_hash`) and removing the plaintext password display.
2. **Hardcoded secret key** (`app.secret_key`) in `app.py` — move this to an environment variable instead of committing it to source.
3. `add_student` and a few other routes don't check `session['staff_id']` — consider adding login checks consistently across all data-modifying routes.
4. `focustracker.db` is committed as a file — for production, exclude it from version control and let `init_db()` create it fresh, or use a managed database.

📄 License

© 2026 Mytheeswaran S. All Rights Reserved.